# 5. DFS

## Aim:

To implement the Depth-First Search (DFS) algorithm using Python to traverse a graph.

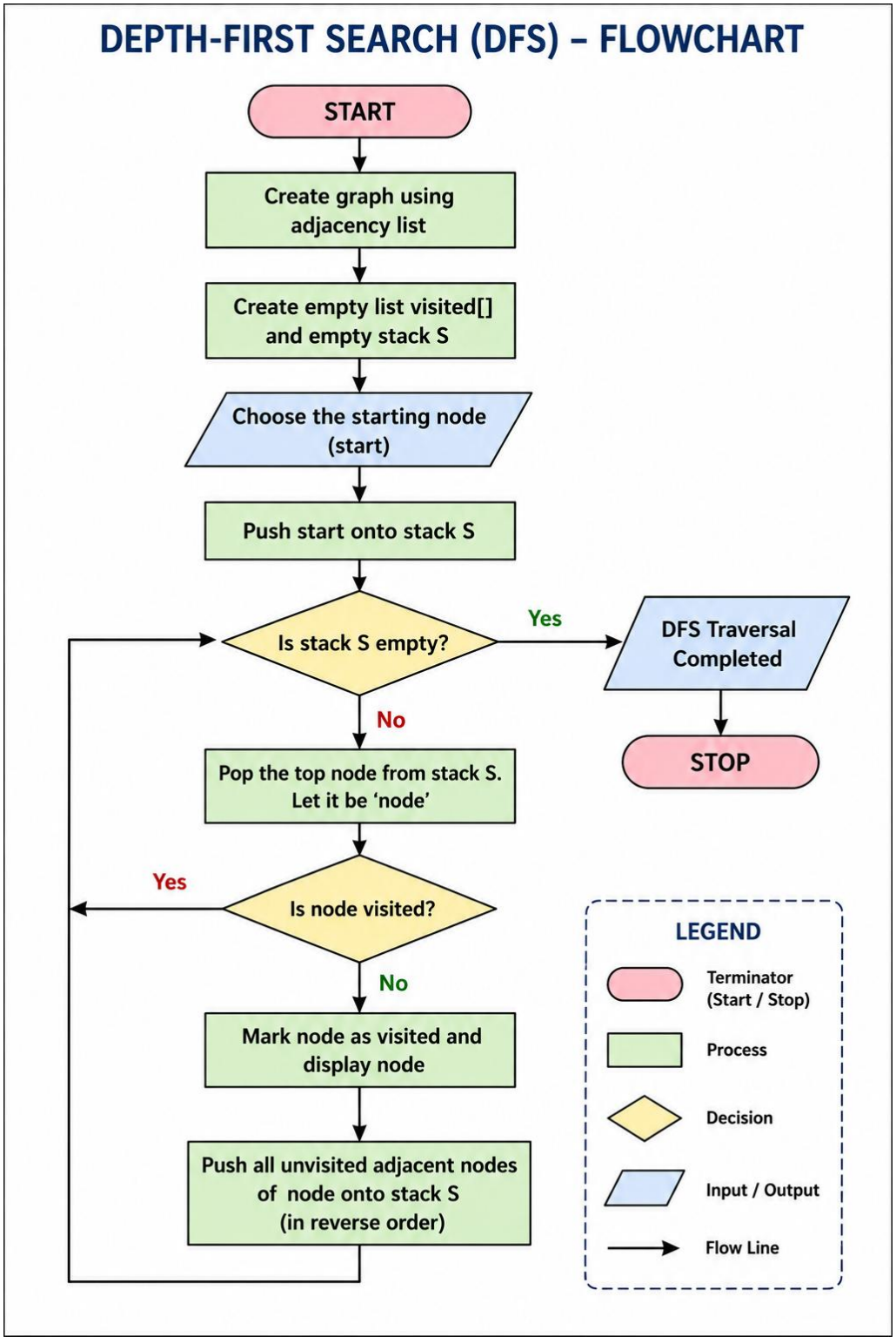
## Objective :

- To understand the Depth-First Search (DFS) algorithm.
- To traverse all the vertices of a graph using a **stack**.
- To visit each node exactly once.
- To display the DFS traversal order.

## Algorithm:

1. Start.
2. Create a graph using an adjacency list.
3. Create an empty **visited** list and an empty **stack**.
4. Choose the starting node.
5. Push the starting node onto the stack.
6. Repeat until the stack becomes empty:
7. Pop the top node from the stack.
8. If the node has not been visited:
    a. Mark it as visited.
    b. Display the node.
    c. Push all unvisited adjacent nodes onto the stack in reverse order.
9. Stop.

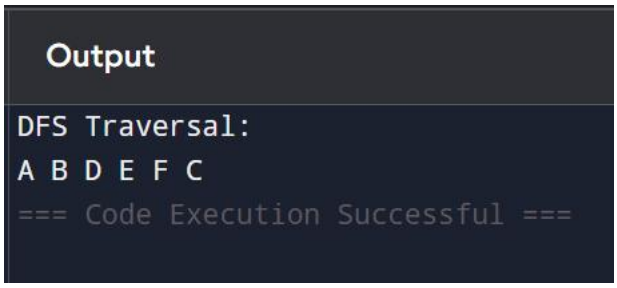
## Flowchart :


DEPTH-FIRST SEARCH (DFS) – FLOWCHART
START
Create graph using adjacency list
Create empty list visited[] and empty stack S
Choose the starting node (start)
Push start onto stack S
Is stack S empty?
Yes
DFS Traversal Completed
STOP
No
Pop the top node from stack S. Let it be 'node'
Is node visited?
Yes
No
Mark node as visited and display node
Push all unvisited adjacent nodes of node onto stack S (in reverse order)
LEGEND
Terminator (Start / Stop)
Process
Decision
Input / Output
Flow Line

## Code:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
visited = []
stack = []
start = 'A'
stack.append(start)
print("DFS Traversal:")
while stack:
    node = stack.pop()
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                stack.append(neighbour)
```

## Output:




## Result :

The Depth-First Search (DFS) algorithm was successfully implemented using Python. The graph was traversed by exploring each branch as deeply as possible before backtracking, and the traversal order was displayed successfully.